

Cryptography 2

Musard Balliu

Lecture 5

DD2395/DD2391

Last Lecture

- **Authentication and authorization**
 - **Authentication - Passwords**
 - **Discretionary Access Control – DAC**
 - **Access control lists**
 - **Capability lists**
 - **Mandatory Access Control - MAC**

Questions?

Today's Plan

- **Mode of Operation for Block Ciphers**
 - ECB, CBC, CTR
- **Asymmetric key cryptography**
 - Diffie-Hellman key exchange
 - RSA

Block Ciphers

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	● One-time pads ● Block ciphers (e.g. AES) ● Block ciphers with chaining modes (e.g. AES-CBC)	● Diffie-Hellman ● RSA Encryption
Integrity, Authentication	● Hash functions ● MACs (e.g. HMAC)	● Digital signatures (e.g. RSA signatures)

Summary: Block Ciphers

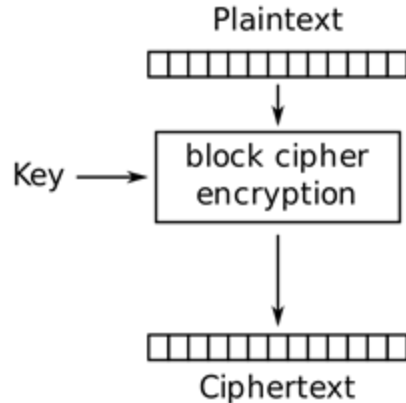
- Encryption: input a k -bit key and n -bit plaintext, receive n -bit ciphertext
- Decryption: input a k -bit key and n -bit ciphertext, receive n -bit plaintext
- Correctness: when the key is fixed, $E(K,M)$ should be bijective
- Security
 - Without the key, $E(K,M)$ is computationally indistinguishable from a random permutation
 - Brute-force attacks take astronomically long and are not possible
- Efficiency: algorithms use XORs and bit-shifting (very fast)
- Implementation: AES is the modern standard
- **Issues**
 - Not IND-CPA secure because they're deterministic
 - Can only encrypt n -bit messages

Block Cipher Modes of Operation

Let's design it together

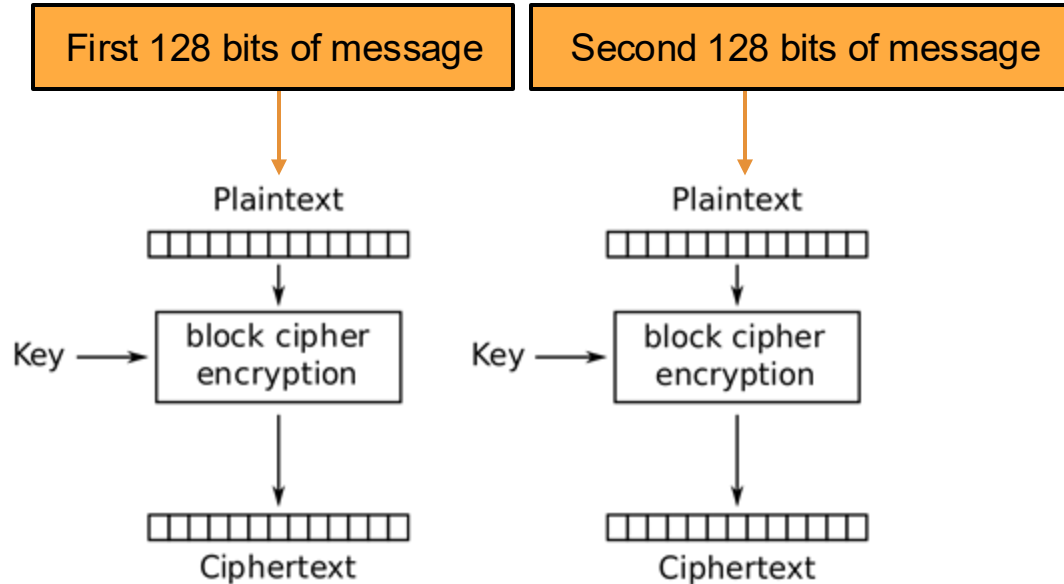
Here's an AES block. Remember that it can only encrypt 128-bit messages.

How can we use AES to encrypt a longer message (say, 256 bits?)



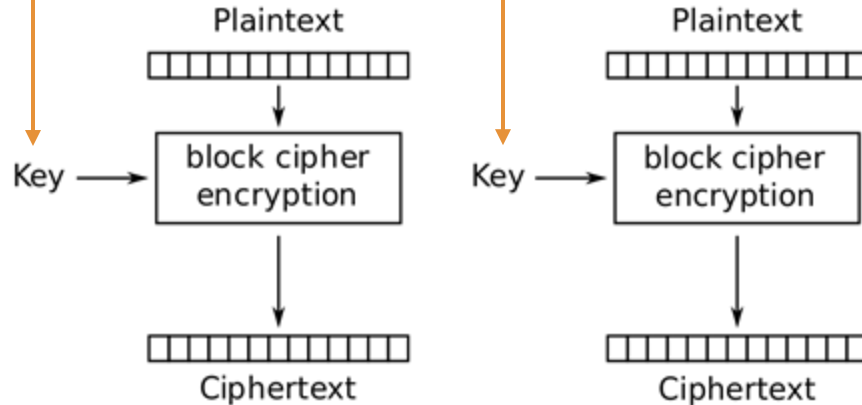
Let's design it together

Idea: Let's use AES twice!



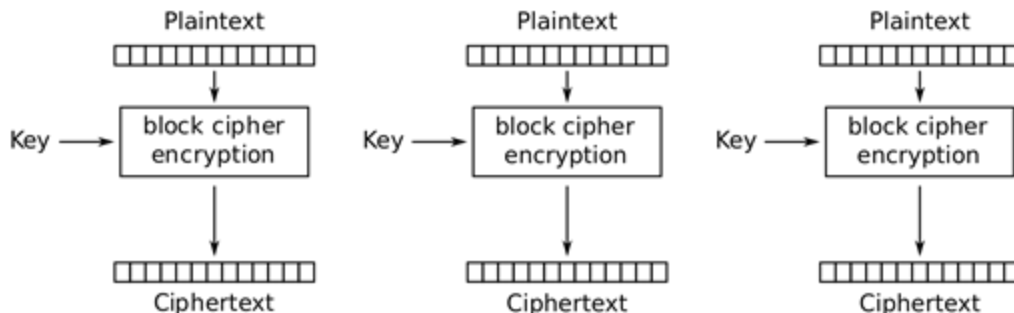
Let's design it together

Note that we are using the same key twice. We want to avoid a situation like one-time pads where we need very long keys.



ECB Mode

- We've just designed **electronic code book (ECB) mode**
 - $\text{Enc}(K, M) = C_1 \parallel C_2 \parallel \dots \parallel C_m$
 - Assume m is the number of blocks of plaintext in M , each of size n
- AES-ECB is not IND-CPA secure. Why?
 - Because ECB is deterministic

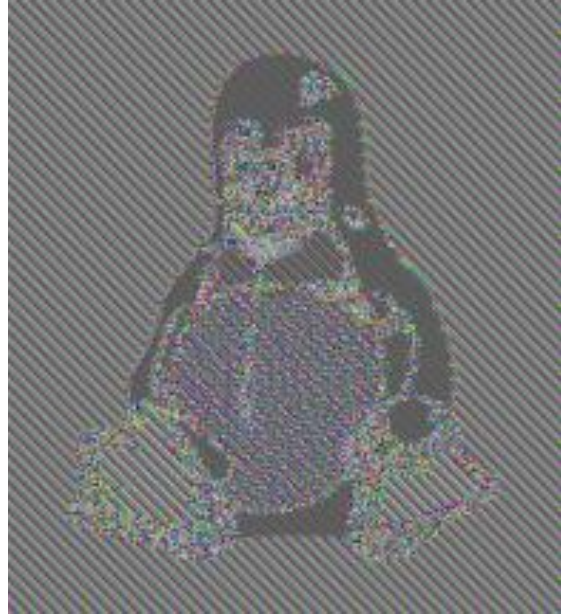


Electronic Codebook (ECB) mode encryption

ECB Mode: Penguin



Original image

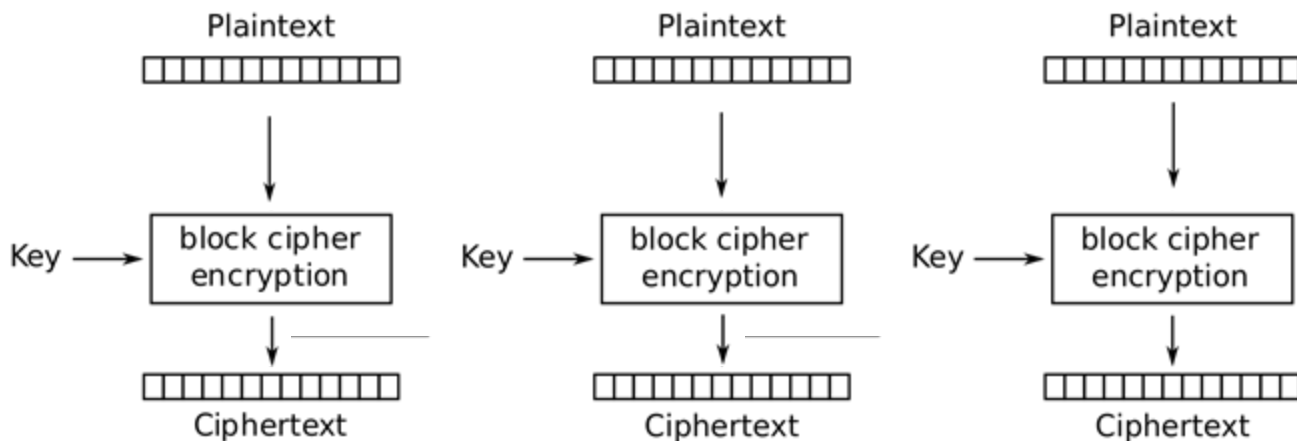


Encrypted with ECB

Let's design it together

Here's ECB mode. It's not IND-CPA secure because it's deterministic.

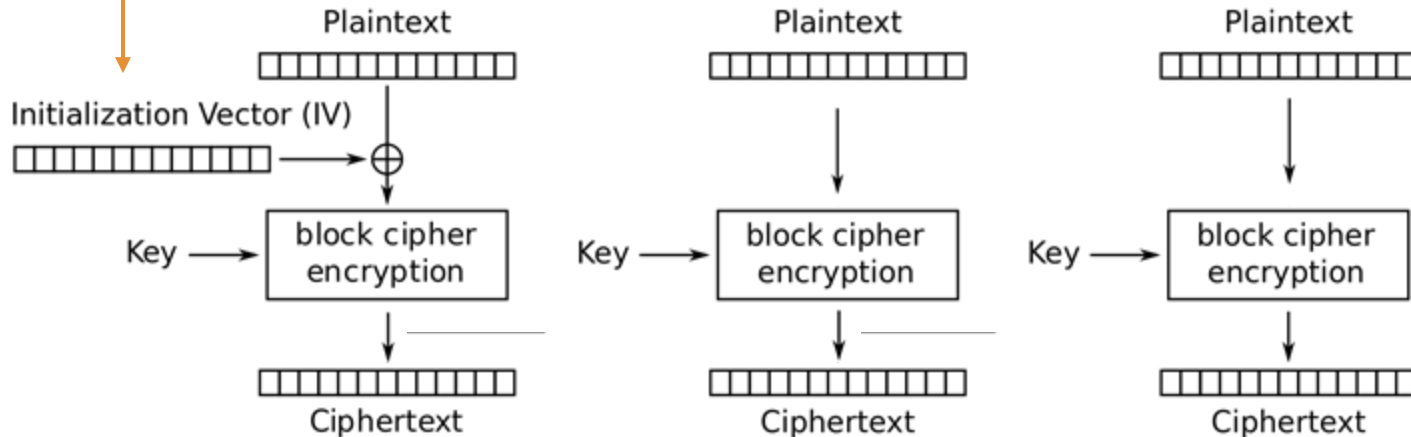
Let's fix that by adding some randomness.



Let's design it together

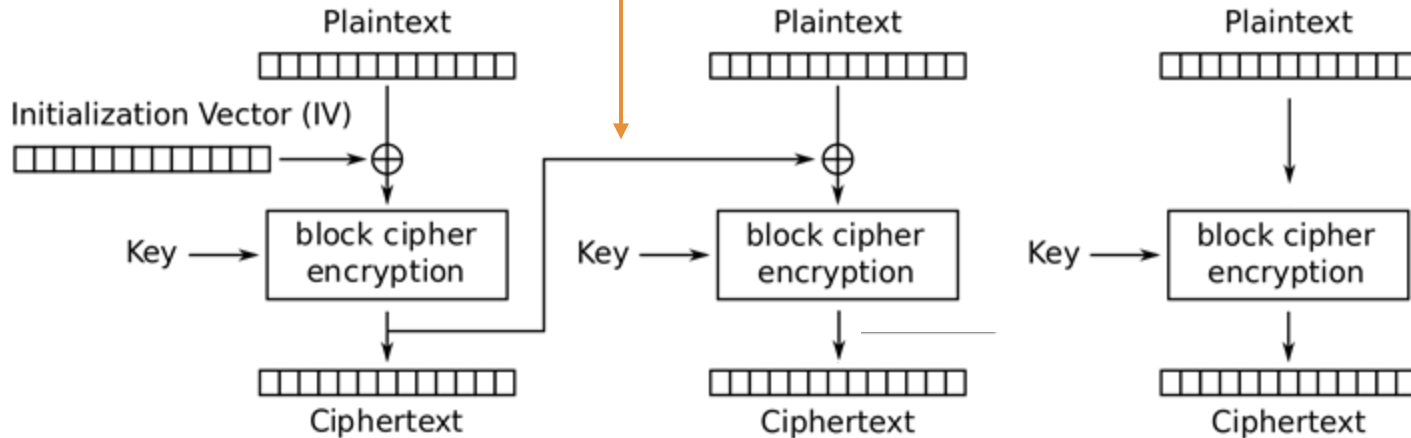
The **Initialization Vector (IV)** is different for every encryption. Now the first ciphertext block will be different for every encryption!

Okay, but the other blocks are still deterministic...

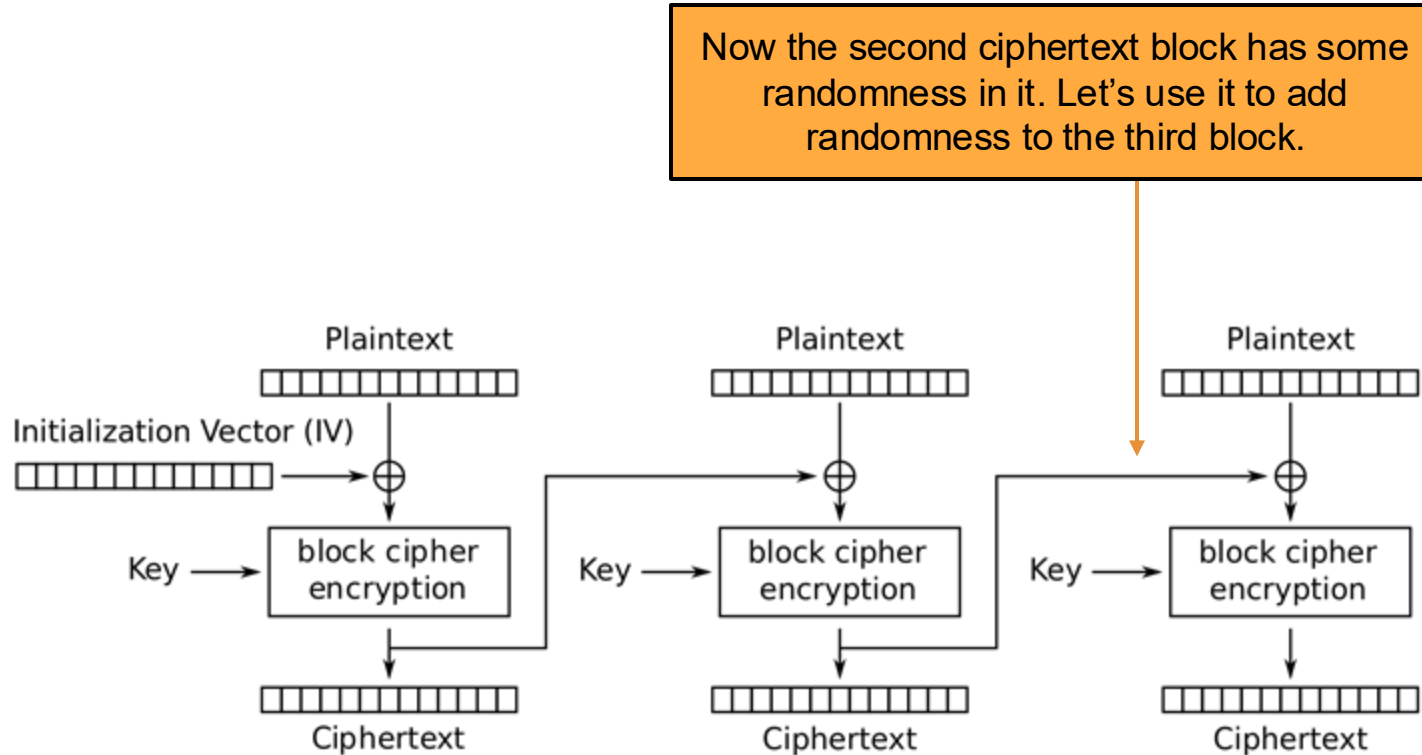


Let's design it together

Idea: The first ciphertext block was computed with some randomness. Let's use it to add randomness to the second block.

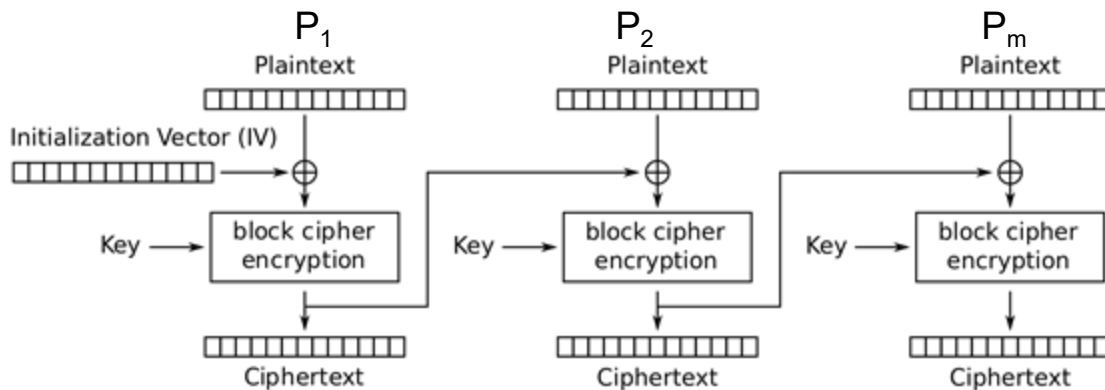


Let's design it together



CBC Mode

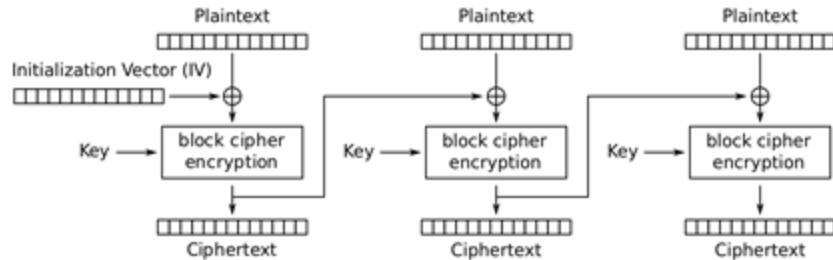
- We've just designed **cipher block chaining (CBC)** mode
- $C_i = Ek(M_i \oplus C_{i-1})$; $C_0 = IV$
- $Enc(K, M)$:
 - Split M in m plaintext blocks $P_1 \dots P_m$ each of size n
 - Choose a random IV
 - Compute and output $(IV, C_1, \dots, C_m)$ as the overall ciphertext
- How do we decrypt?



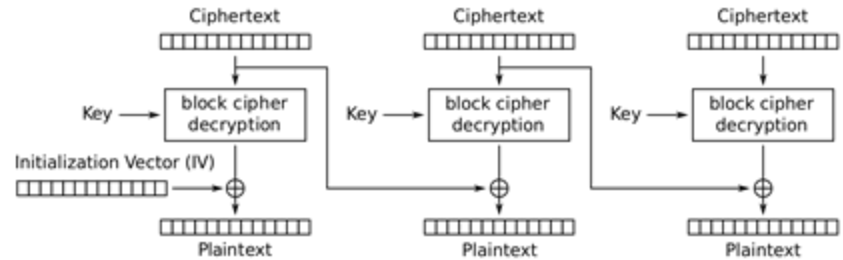
Cipher Block Chaining (CBC) mode encryption

CBC Mode: Decryption

- How do we decrypt CBC mode?
 - Parse ciphertext as $(IV, C_1, \dots, C_m)$
 - Decrypt each ciphertext and then XOR with IV or previous ciphertext



Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

CBC Mode: Decryption

$$C_i = E_K(M_i \oplus C_{i-1})$$

Definition of encryption

$$D_K(C_i) = D_K(E_K(M_i \oplus C_{i-1}))$$

Decrypting both sides

$$D_K(C_i) = M_i \oplus C_{i-1}$$

Decryption and encryption cancel

$$D_K(C_i) \oplus C_{i-1} = M_i \oplus C_{i-1} \oplus C_{i-1}$$

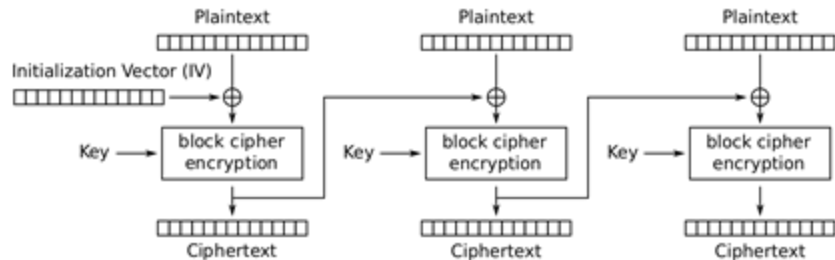
XOR both sides with C_{i-1}

$$D_K(C_i) \oplus C_{i-1} = M_i$$

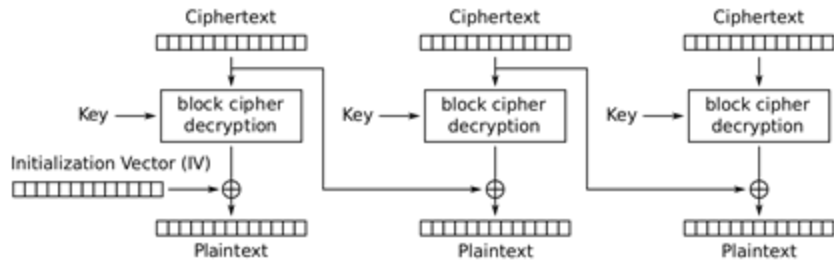
XOR property

CBC Mode: Efficiency & Parallelism

- Can encryption be parallelized?
 - No, we have to wait for block i to finish before encrypting block $i+1$
- Can decryption be parallelized?
 - Yes, decryption only requires ciphertext as input



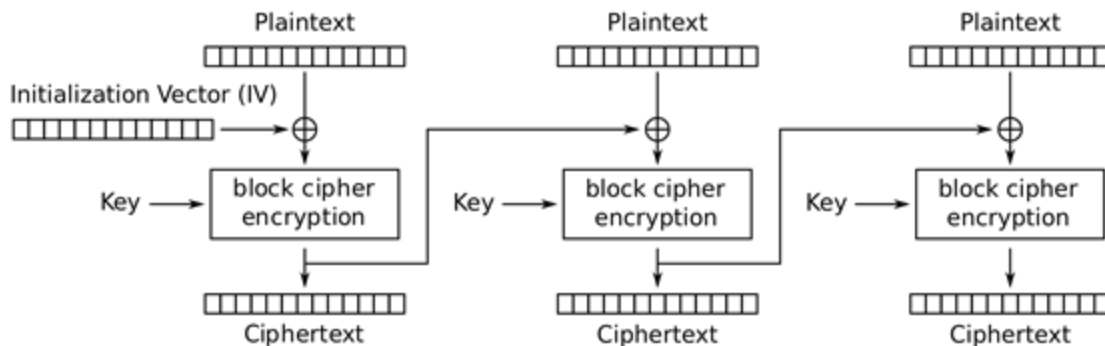
Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

CBC Mode: Padding

- What if you want to encrypt a message that isn't a multiple of the block size?
 - AES-CBC is only defined if the plaintext length is a multiple of the block size
- Solution: Pad the message until it's a multiple of the block size
 - **Padding:** Adding dummy bytes at the end of the message until it's the proper length



Cipher Block Chaining (CBC) mode encryption

CBC Mode: Padding

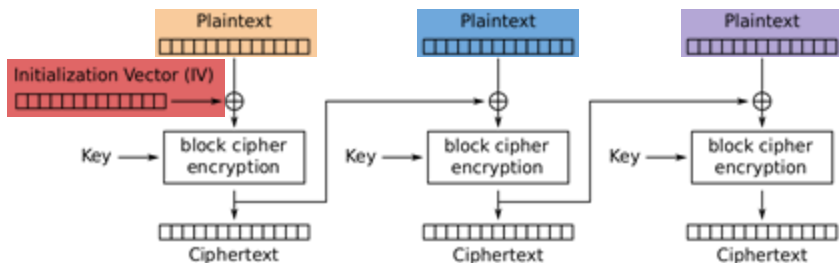
- What padding scheme should we use?
 - Padding with 0's?
 - Doesn't work: What if our message already ends with 0's?
 - Padding with 1's?
 - Same problem
- We need a scheme that can be unpadded without ambiguity
 - One scheme that works: Append a 1, then pad with 0's
 - If plaintext is multiple of n , you still need to pad with an entire block
 - Another scheme: Pad with the number of padding bytes
 - So if you need 1 byte, pad with **01**; if you need 3 bytes, pad with **03 03 03**
 - If you need 0 padding bytes, pad an entire dummy block
 - This is called PKCS #7

CBC Mode: Security

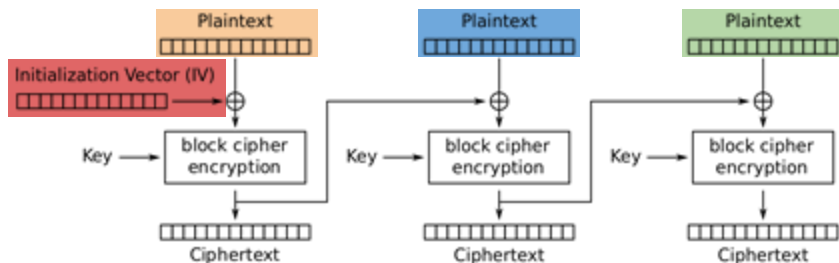
- AES-CBC is IND-CPA secure. With what assumption?
 - The IV must be randomly generated and never reused
- What happens if you reuse the IV?
 - The scheme becomes deterministic: No more IND-CPA security

CBC Mode: IV Reuse

- Consider two three-block messages: $P_1P_2P_3$ and $P_1P_2P_4$
 - The first two blocks are the same for both messages, but the last block is different
 - What if we encrypt them with the same IV?
- When the IV is reused, CBC mode reveals when two messages start with the same plaintext blocks, up to the first different plaintext block



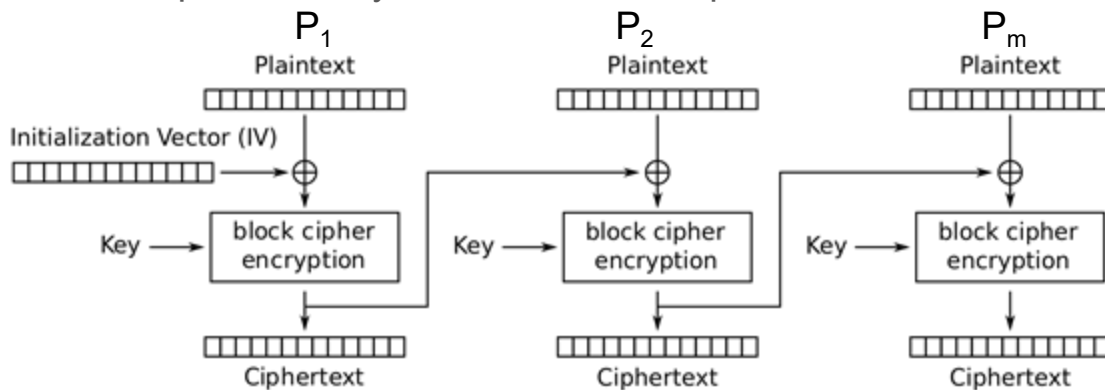
Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode encryption

CBC Mode is IND-CPA (when used correctly)

- $\text{Enc}(K, M)$:
 - Split M in m plaintext blocks $P_1 \dots P_m$ each of size n
 - Choose random IV, compute and output $(IV, C_1, \dots, C_m)$ as the overall ciphertext
- Why IND-CPA?
 - If there exists an attacker that wins in the IND-CPA game, then there exists an attacker that breaks the block cipher security. Proof is out of scope.



Cipher Block Chaining (CBC) mode encryption

CBC Mode: Penguin



Original image



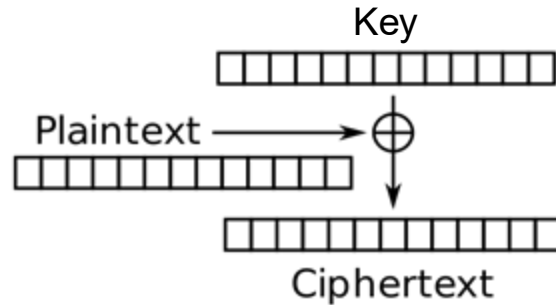
Encrypted with CBC, with random IVs

ECB and CBC: Summary

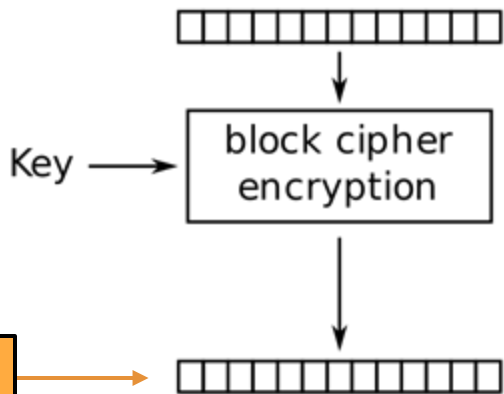
- ECB mode: Deterministic, so not IND-CPA secure
- CBC mode
 - IND-CPA secure, assuming no IV reuse
 - Encryption is not parallelizable
 - Decryption is parallelizable
 - Must pad plaintext to a multiple of the block size
 - IV reuse leads to leaking the existence of identical blocks at the start of the message

CTR (Counter) Mode: Let's design it together

One-time pads are secure if we never reuse the key.



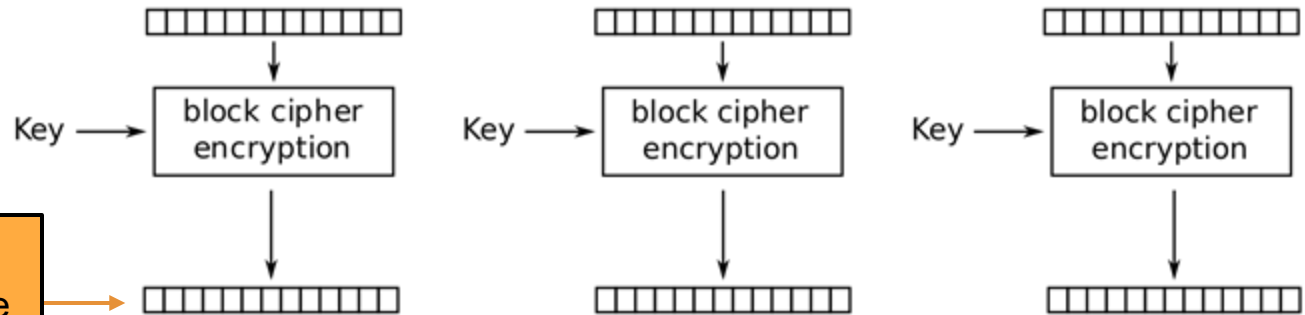
CTR (Counter) Mode: Let's design it together



If the attacker doesn't know the key, the block cipher output looks random.

New idea: Can we use block ciphers to simulate a one-time pad?

CTR (Counter) Mode: Let's design it together

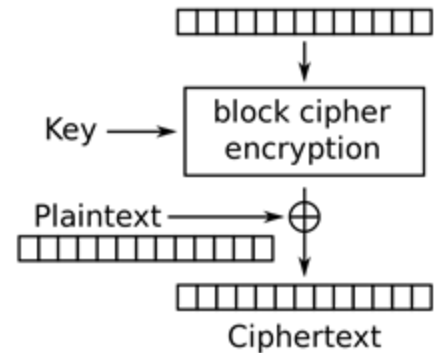
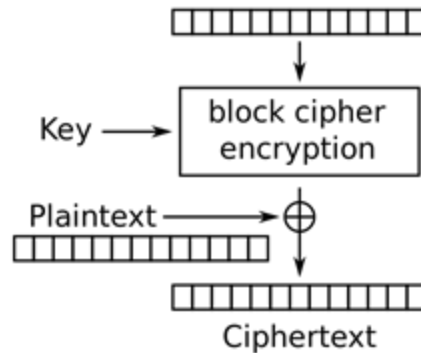
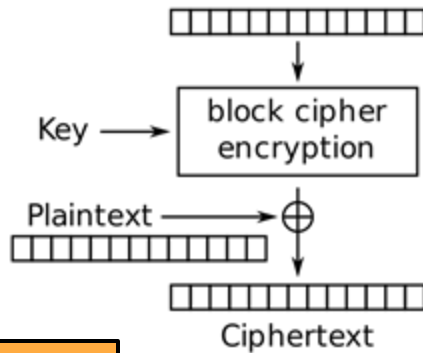


If the attacker doesn't know the key, all of these outputs look random.

CTR (Counter) Mode: Let's design it together

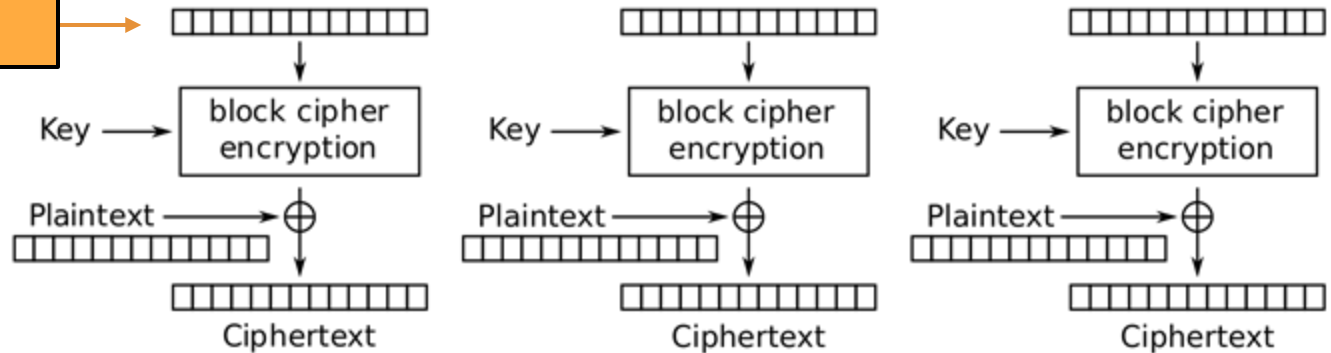
Idea: Use this random-looking output as a one-time pad!

Remember one-time pads: XOR the pad with plaintext to get ciphertext



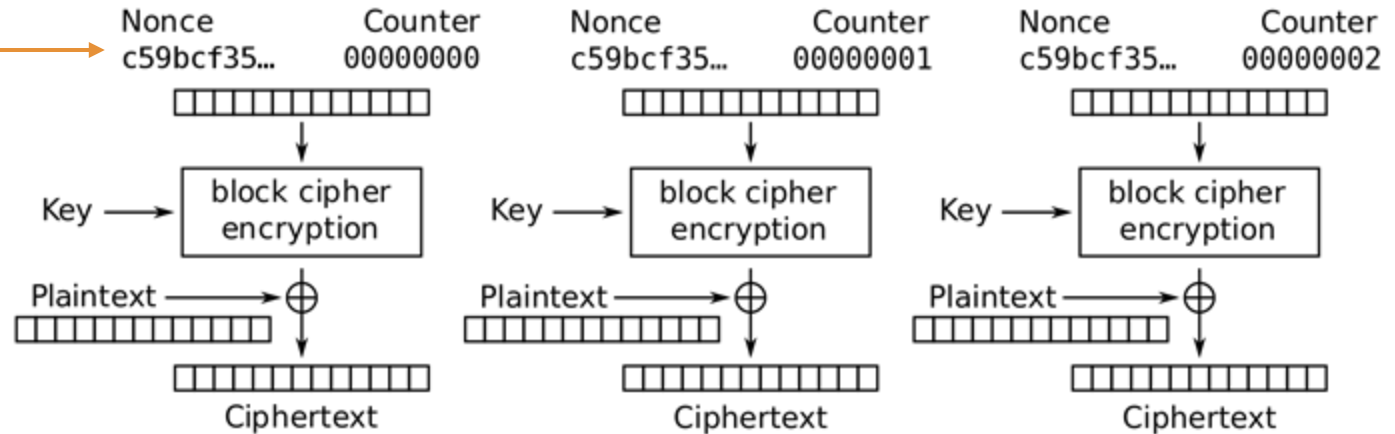
CTR (Counter) Mode: Let's design it together

What do we use as input to the block cipher?



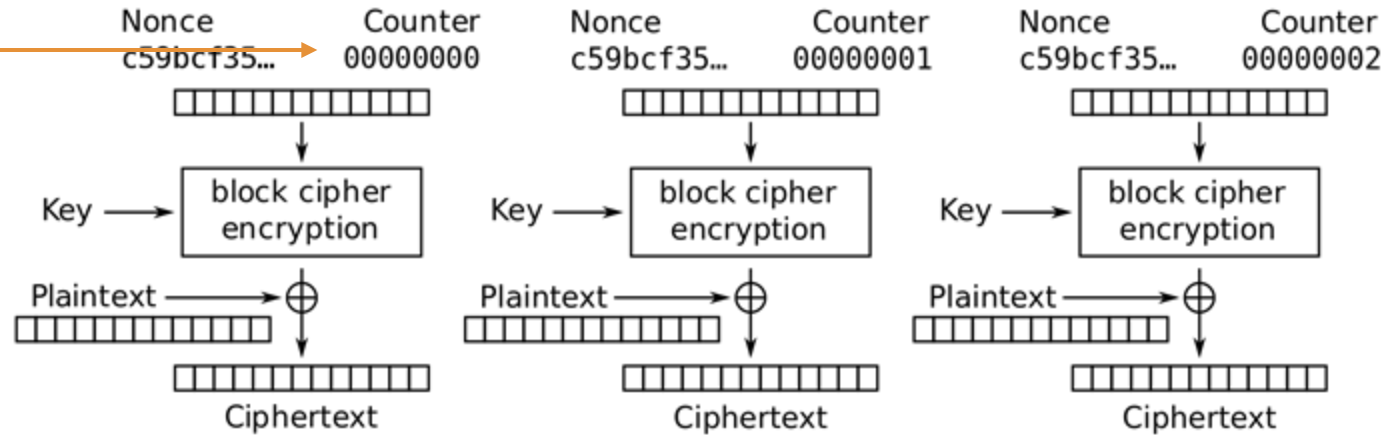
CTR (Counter) Mode: Let's design it together

IND-CPA schemes need randomness, so let's put a random **nonce** here!



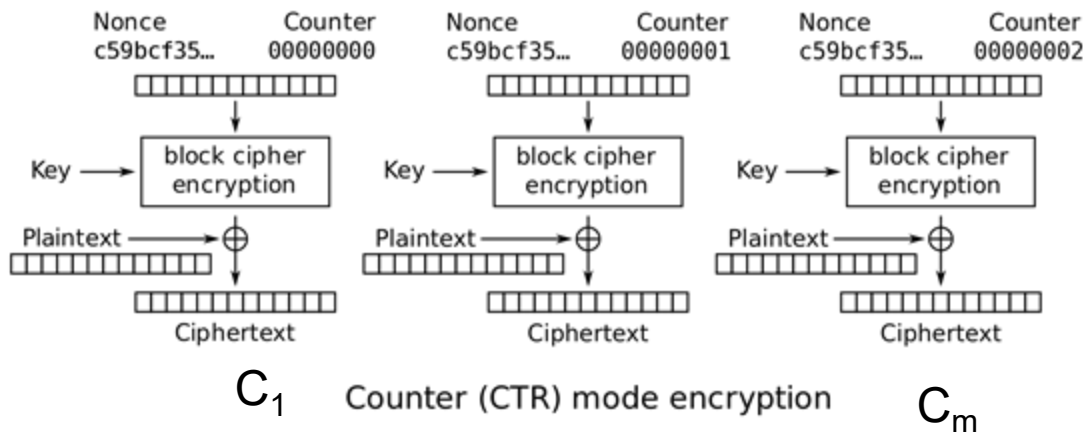
CTR (Counter) Mode: Let's design it together

The **counter** increments per block to ensure each block cipher output is different.



CTR (Counter) Mode

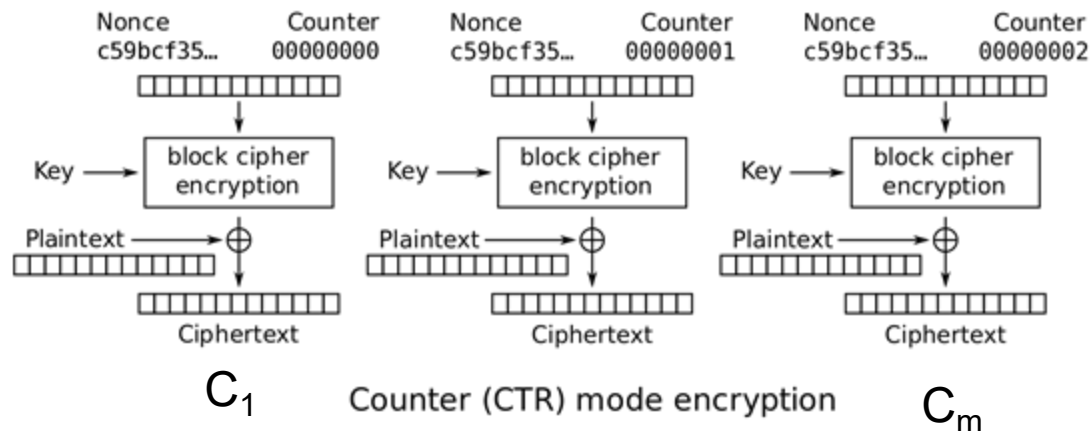
- Note: the random value is named the nonce here, but the idea is the same as the IV in CBC mode
- Overall ciphertext is (Nonce, $C_1, \dots, C_m$)



CTR Mode

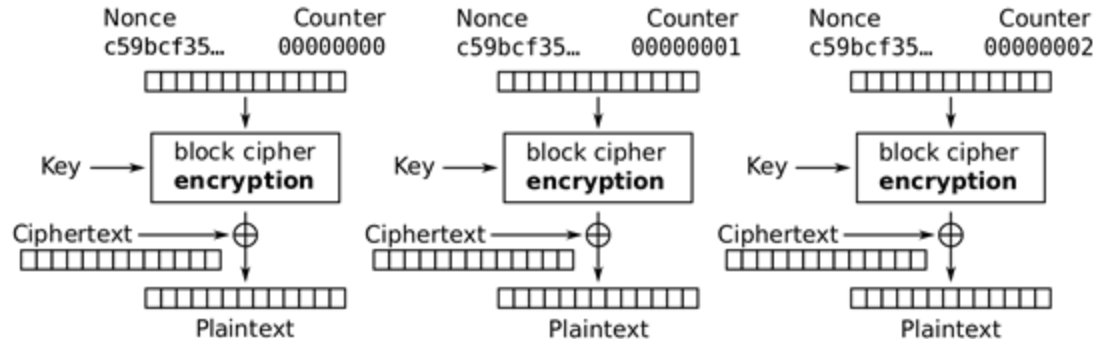
- $\text{Enc}(K, M)$:
 - Split M in plaintext blocks $P_1 \dots P_m$ (each of block size n)
 - Choose random nonce
 - Compute and output $(\text{Nonce}, C_1, \dots, C_m)$

How do you decrypt?



CTR Mode: Decryption

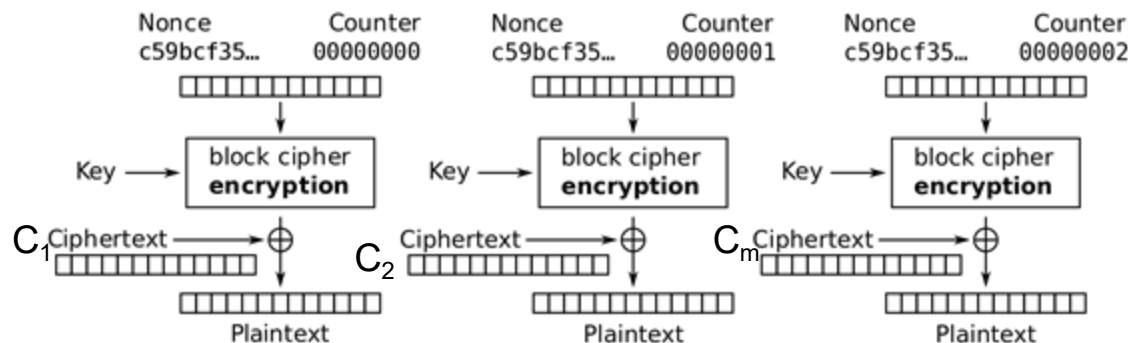
- Recall one-time pad: XOR with ciphertext to get plaintext
- Note: we are only using block cipher encryption, not decryption



Counter (CTR) mode decryption

CTR Mode: Decryption

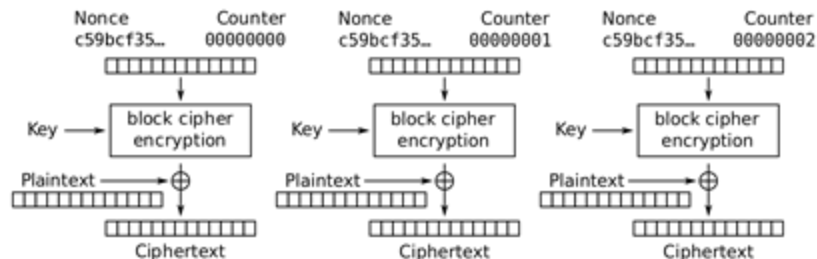
- $\text{Dec}(K, C)$:
 - Parse C into $(\text{nonce}, C_1, \dots, C_m)$
 - Compute P_i by XORing C_i with output of E_k on nonce and counter
 - Concatenate resulting plaintexts and output $M = P_1 \dots P_m$



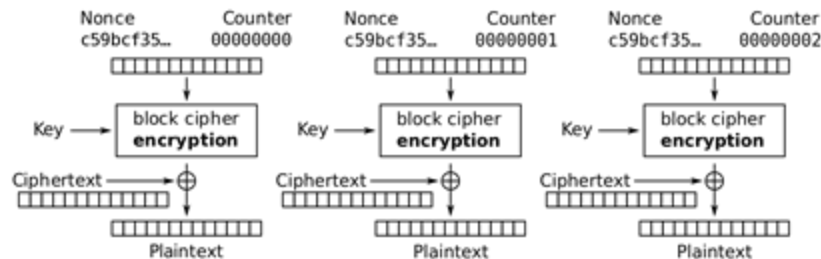
Counter (CTR) mode decryption

CTR Mode: Efficiency

- Can encryption be parallelized?
 - Yes
- Can decryption be parallelized?
 - Yes



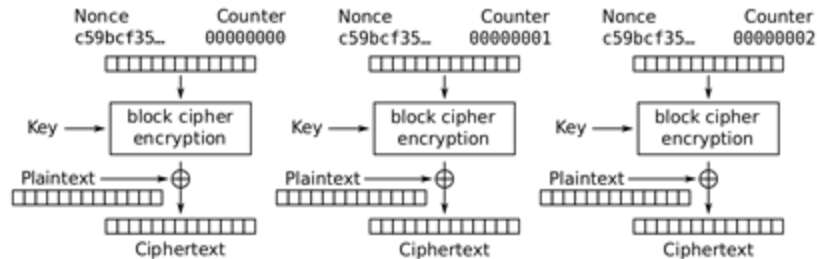
Counter (CTR) mode encryption



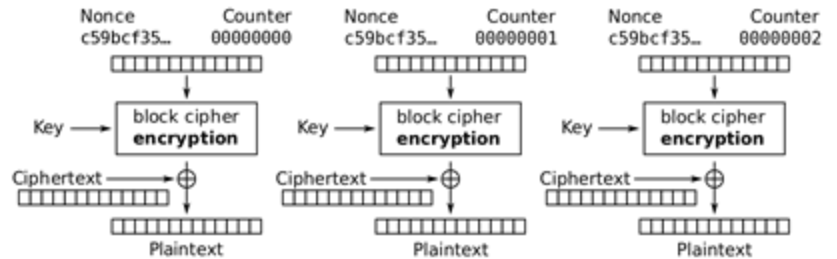
Counter (CTR) mode decryption

CTR Mode: Padding

- Do we need to pad messages?
 - No! We can just cut off the parts of the XOR that are longer than the message.



Counter (CTR) mode encryption



Counter (CTR) mode decryption

CTR Mode: Security

- AES-CTR is IND-CPA secure. With what assumption?
- The nonce must be randomly generated and never reused
- What happens if you reuse the nonce?
- Equivalent to reusing a key in a one-time pad
 - Recall: Key reuse in a one-time pad is catastrophic: usually leaks enough information for an attacker to deduce the entire plaintext

CTR Mode: Penguin



Original image



Encrypted with CTR, with random nonces

IVs and Nonces

- **Initialization vector (IV):** A random, but public, one-use value to introduce randomness into the algorithm
 - For CTR mode, we say that you use a **nonce** (number used once), since the value has to be unique, not necessarily random.
 - We use IV and nonce interchangeably
- **Never reuse IVs**
 - In some algorithms, IV/nonce reuse leaks limited information (e.g. CBC)
 - In some algorithms, IV/nonce reuse leads to catastrophic failure (e.g. CTR)

Comparing Modes of Operation

- If you need high performance, which mode is better?
 - CTR mode, because you can parallelize both encryption and decryption
- If you're paranoid about security, which mode is better?
 - CBC mode is better
- Theoretically, CBC and CTR mode are equally secure if used properly
 - However, if used improperly (IV/nonce reuse), CBC only leaks partial information, and CTR fails catastrophically

Do Block Ciphers provide Integrity and Authenticity?

- Block ciphers are designed for *confidentiality* (IND-CPA)
- If an attacker tampers with the ciphertext, we are not guaranteed to detect it
- Remember Mallory: An *active* manipulator who wants to tamper with the message

Lack of Integrity and Authenticity

- Consider CTR mode
- What if Mallory tampers with the ciphertext using XOR?

	P	a	y		M	a	l		\$	1	0	0
M	0x50	0x61	0x79	0x20	0x4d	0x61	0x6c	0x20	0x24	0x31	0x30	0x30
$\oplus$												
$E_K(i)$	0x8a	0xe3	0x5e	0xcf	0x3b	0x40	0x46	0x57	0xb8	0x69	0xd2	0x96
$=$												
C	0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x58	0xe2	0xa6

Lack of Integrity and Authenticity

- Suppose Mallory knows the message M
- How can Mallory change the M to say **Pay Mal \$900?**

M	P	a	y		M	a	l		\$	1	0	0
	0x50	0x61	0x79	0x20	0x4d	0x61	0x6c	0x20	0x24	0x31	0x30	0x30
$\oplus$												
$E_K(i)$	0x8a	0xe3	0x5e	0xcf	0x3b	0x40	0x46	0x57	0xb8	0x69	0xd2	0x96
=												
C	0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x58	0xe2	0xa6

Lack of Integrity and Authenticity

$C_i = M_i \oplus \text{Pad}_i$	$0\mathbf{x58} = 0\mathbf{x31} \oplus \text{Pad}_i$	Definition of CTR																					
$\text{Pad}_i = M_i \oplus C_i$	$\text{Pad}_i = 0\mathbf{x58} \oplus 0\mathbf{x31}$ $= 0\mathbf{x69}$	Solve for the i th byte of the pad																					
$C'_i = M'_i \oplus \text{Pad}_i$	$C'_i = 0\mathbf{x39} \oplus 0\mathbf{x69}$ $= 0\mathbf{x50}$	Compute the changed i th byte																					
C	<table><tr><td>0xda</td><td>0x82</td><td>0x27</td><td>0xef</td><td>0x76</td><td>0x21</td><td>0x2a</td><td>0x77</td><td>0x9c</td><td>0x58</td><td>0xe2</td><td>0xa6</td></tr></table>											0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x58	0xe2	0xa6
0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x58	0xe2	0xa6												
C'	<table><tr><td>0xda</td><td>0x82</td><td>0x27</td><td>0xef</td><td>0x76</td><td>0x21</td><td>0x2a</td><td>0x77</td><td>0x9c</td><td>0x50</td><td>0xe2</td><td>0xa6</td></tr></table>											0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x50	0xe2	0xa6
0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x50	0xe2	0xa6												

Lack of Integrity and Authenticity

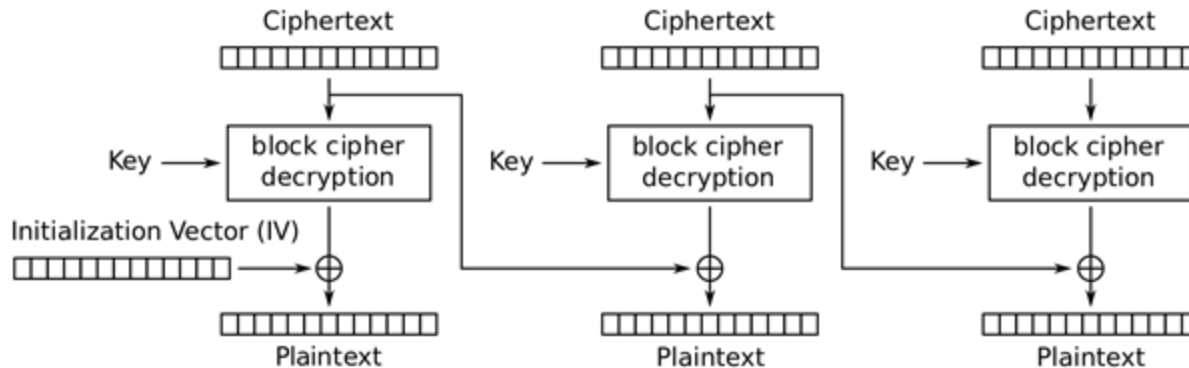
- What happens when we decrypt C' ?
 - The message looks like “Pay Mal \$900” now!
 - Note: Mallory didn’t have to know the key; no integrity or authenticity for CTR mode!

C'	0xda	0x82	0x27	0xef	0x76	0x21	0x2a	0x77	0x9c	0x50	0xe2	0xa6
$\oplus$												
$E_K(i)$	0x8a	0xe3	0x5e	0xcf	0x3b	0x40	0x46	0x57	0xb8	0x69	0xd2	0x96
$=$												
P'	0x50	0x61	0x79	0x20	0x4d	0x61	0x6c	0x20	0x24	0x39	0x30	0x30
	P	a	y		M	a	l		\$	9	0	0

Lack of Integrity and Authenticity

- What about CBC?

- Altering a bit of the ciphertext causes some blocks to become random gibberish
- However, Bob doesn't know that Alice did not send random gibberish, so it still does *not* provide integrity or authenticity



Cipher Block Chaining (CBC) mode decryption

Public-Key Cryptography

Public-Key Cryptography

- In public-key schemes, each person has two keys
 - **Public key:** Known to everybody
 - **Private key:** Only known by that person
 - Keys come in pairs: every public key corresponds to one private key
- Uses number theory
 - Examples: Modular arithmetic, factoring, discrete logarithm problem
 - Contrast with symmetric-key cryptography (uses XORs and bit-shifts)
- Messages are numbers
 - Contrast with symmetric-key cryptography (messages are bit strings)
- Benefit: No longer need to assume that Alice and Bob already share a secret
- Drawback: Much slower than symmetric-key cryptography
 - Number theory calculations are much slower than XORs and bit-shifts

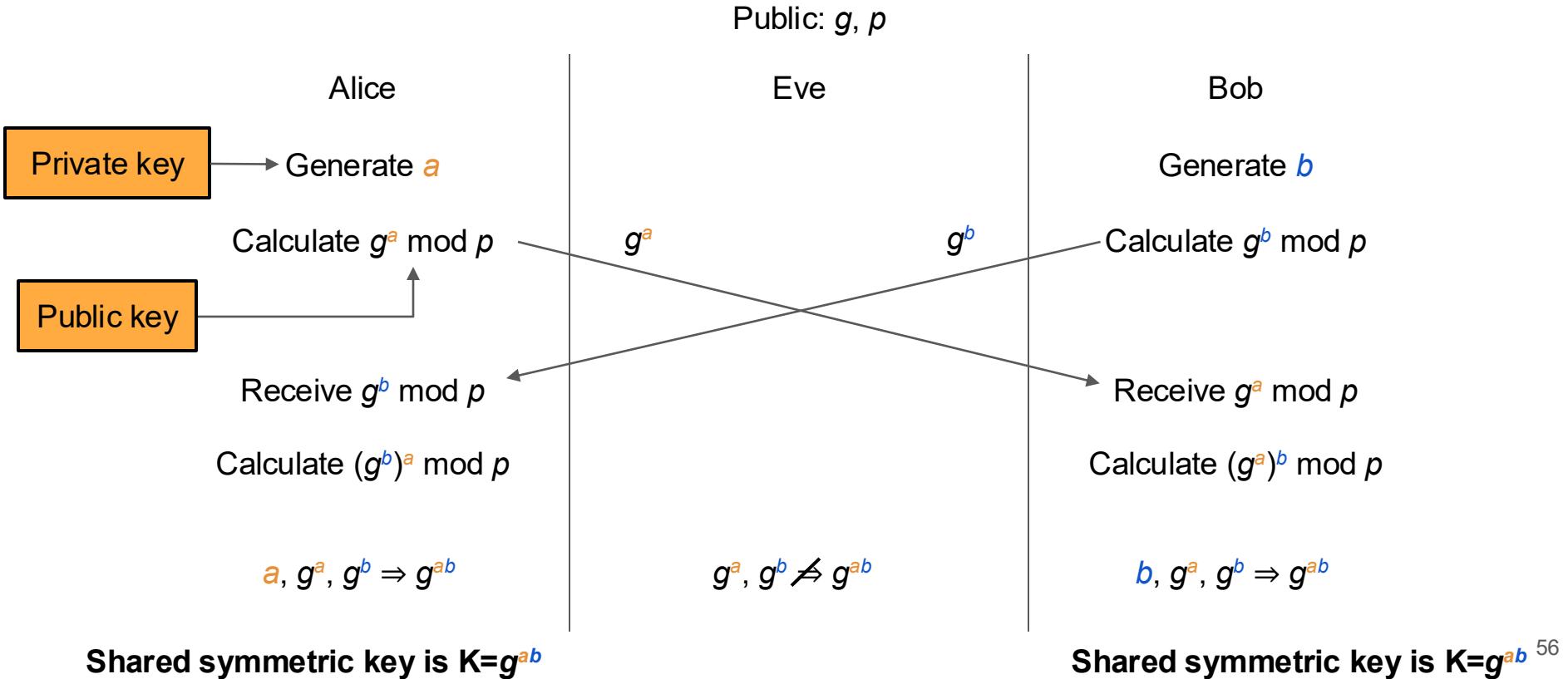
Diffie-Hellman Key Exchange

How do Alice and Bob exchange keys?

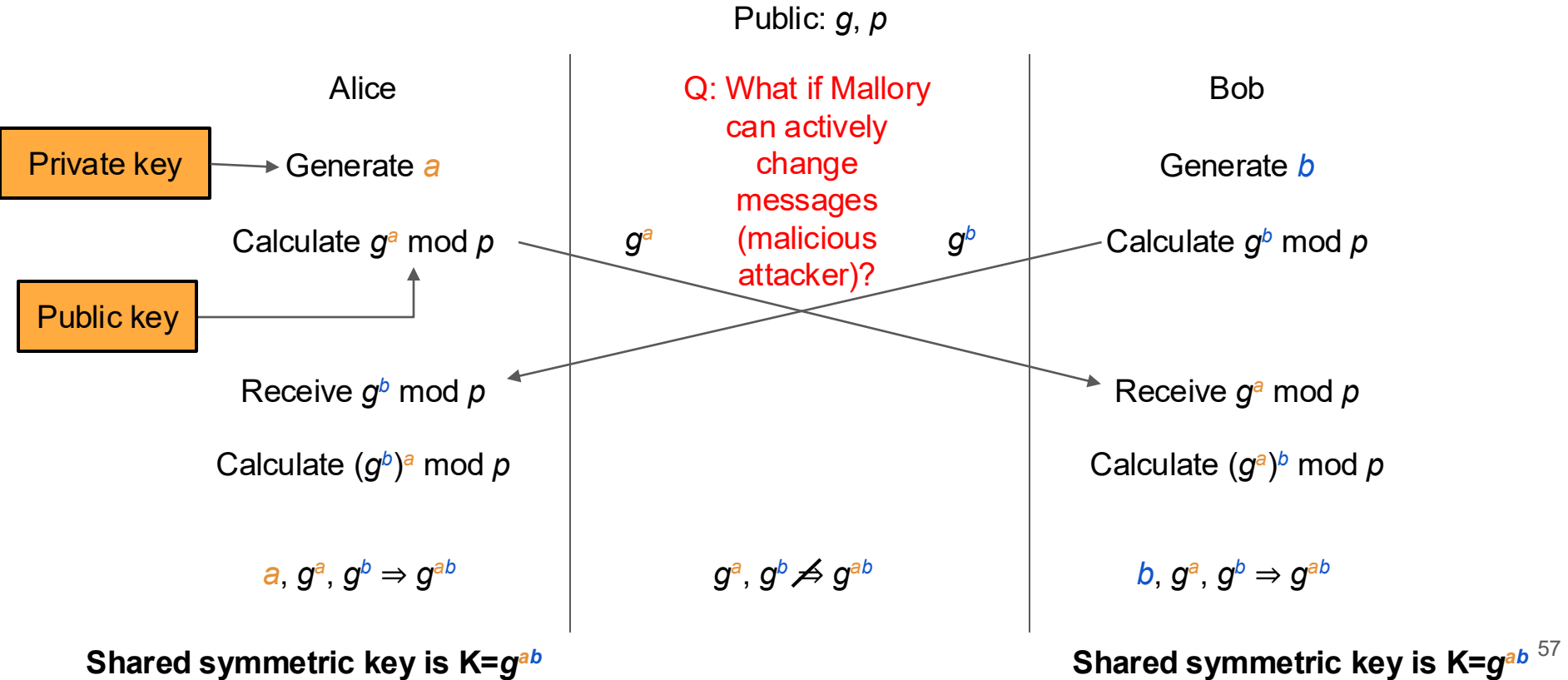
Discrete Log Problem and Diffie-Hellman Problem

- Assume everyone knows a large prime p (e.g. 2048 bits long) and a generator g
 - Don't worry about what a generator is
- **Discrete logarithm problem (discrete log problem):** Choose a long random number a (e.g., 2048 bits), given g , p , $g^a \bmod p$: it is computationally hard to find a
- **Diffie-Hellman assumption:** Choose long random numbers a and b (e.g., 2048 bits), given g , p , $g^a \bmod p$, and $g^b \bmod p$, no polynomial time attacker can distinguish between a random value R and $g^{ab} \bmod p$.
 - Note: Multiplying the values doesn't work, since you get $g^{a+b} \bmod p \neq g^{ab} \bmod p$

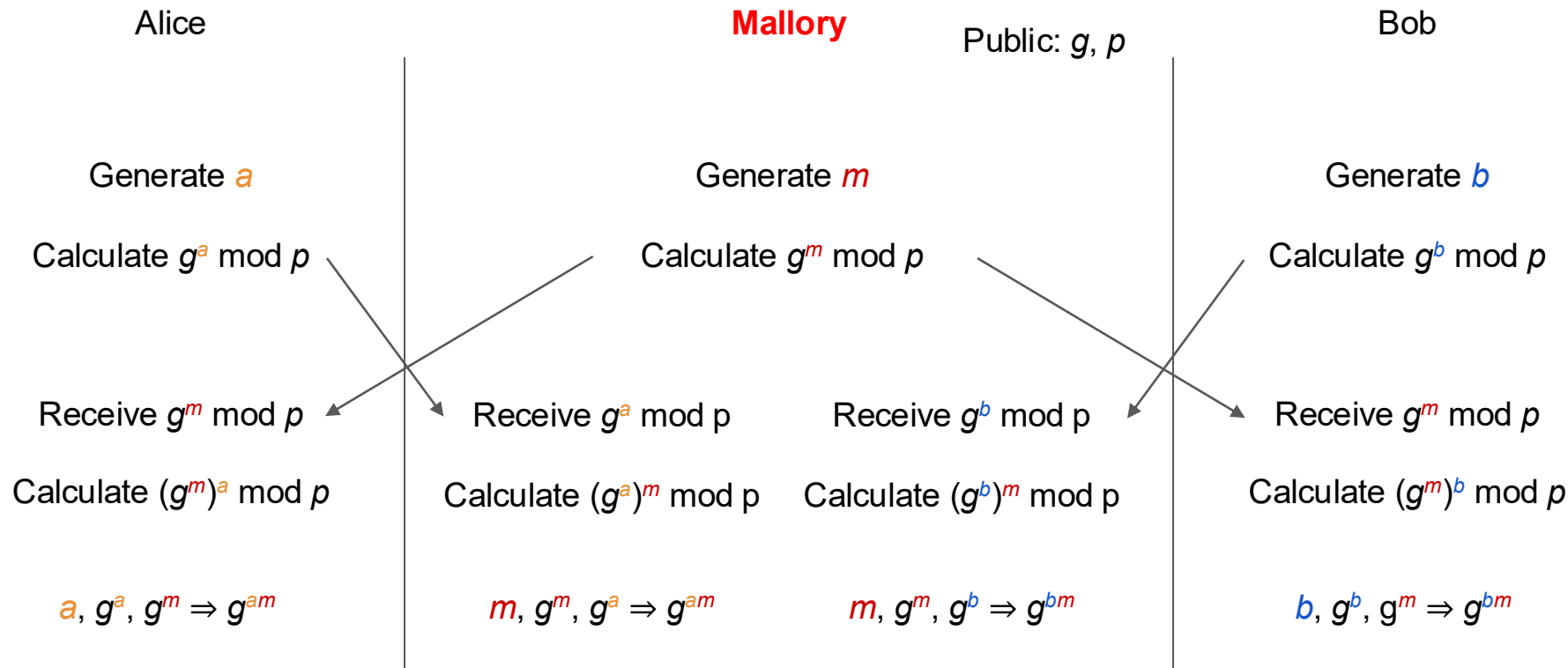
Diffie-Hellman Key Exchange



Diffie-Hellman Key Exchange



Diffie-Hellman: Man-in-the-middle attack



Diffie-Hellman: Issues

- Diffie-Hellman is not secure against a MITM adversary
- DHE is an *active protocol*: Alice and Bob need to be online at the same time to exchange keys
 - What if Bob wants to encrypt something and send it to Alice for her to read later?
- Diffie-Hellman does not provide *authentication*
 - You exchanged keys with someone, but Diffie-Hellman makes no guarantees about who you exchanged keys with; it could be Mallory!
 - Can provide authenticity in combination with digital signatures

Summary: Diffie-Hellman Key Exchange

- Algorithm:
 - Alice chooses a and sends $g^a \bmod p$ to Bob
 - Bob chooses b and sends $g^b \bmod p$ to Alice
 - Their shared secret is $(g^a)^b = (g^b)^a = g^{ab} \bmod p$
- Issues
 - *Not* secure against MITM
 - Both parties must be online
 - Does not provide authenticity
- Some issue are solved by ElGamal encryption
 - Not covered

RSA Encryption

RSA Encryption: Definition

- **KeyGen():**
 - Randomly pick two large primes, p and q
 - Done by picking random numbers and then using a test to see if the number is (probably) prime
 - Compute $N = pq$
 - N is usually between 2048 bits and 4096 bits long
 - Choose e
 - Requirement: e is relatively prime to $(p - 1)(q - 1)$
 - Requirement: $2 < e < (p - 1)(q - 1)$
 - Compute $d = e^{-1} \bmod (p - 1)(q - 1)$
 - Algorithm: Extended Euclid's algorithm
 - **Public key:** N and e
 - **Private key:** d

RSA Encryption: Definition

- $\text{Enc}(e, N, M)$:
 - Output $M^e \bmod N$
- $\text{Dec}(d, C)$:
 - Output $C^d = (M^e)^d \bmod N$

RSA Encryption: Security

- **RSA problem:** Given large $N = pq$ and $C = M^e \bmod N$, it is hard to find M
 - No harder than the factoring problem (if you can factor N , you can recover d)
- Current best solution is to factor N , but unknown whether there is an easier way
 - Factoring problem is assumed to be hard

Hybrid Encryption

- Issues with public-key encryption
 - Notice: We can only encrypt small messages because of the modulo operator
 - Notice: There is a lot of math, and computers are slow at math
 - Result: Asymmetric doesn't work for large messages
- **Hybrid encryption:** Encrypt data under a randomly generated key K using symmetric encryption, and encrypt K using asymmetric encryption
 - Benefit: Now we can encrypt large amounts of data quickly using symmetric encryption, and we still have the security of asymmetric encryption
- Almost all cryptographic systems use hybrid encryption

Homework

- Check out the reading material